Fitting Models

Saturday, February 28, 2026 12:12 AM

- We consider how to find the parameter values that minimize the loss.
  - ↳ Known as learning/training/fitting the model.
- General Process:
  1) Choose initial parameter values
  2) Compute the gradients of the loss with respect to the parameters.
  3) Adjust the parameters based on the gradients to decrease the loss. (↻ Repeat)
- This chapter considers algorithms that adjust parameters to decrease the loss.

6.1: Gradient Descent:

- We need a training set $\{\bar{x}_i, \bar{y}_i\}$ of input/output pairs.
- We seek parameters ϕ for the model $f[\bar{x}_i, \phi]$ that maps inputs $\bar{x}_i$ to outputs $\bar{y}_i$ as close as possible.
- The loss function $L[\phi]$ returns a single number that quantifies the mismatch in this mapping.
- An optimization algorithm finds parameters $\hat{\phi}$ that minimize the loss:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}}\left[L[\phi]\right]$$

- The simplest iterative optimization algorithm is gradient descent.
  - ↳ Starts with initial parameters $\phi = [\phi_0, \phi_1, \cdots, \phi_N]^T$
  - ↳ Iterates:
    1) Compute the derivatives of the loss with respect to the parameters

$$\frac{\partial L}{\partial \phi} = \begin{bmatrix} \partial L/\partial \phi_0 \\ \partial L/\partial \phi_1 \\ \vdots \\ \partial L/\partial \phi_N \end{bmatrix}$$

    2) Update the parameters with the rule:

$$\phi \leftarrow \phi - \alpha \cdot \frac{\partial L}{\partial \phi}$$

where α ↳ Determines the magnitude of the change

- The first step determines the fastest rate of change up the loss function.
- The second step moves a distance α down with said rate of change.

} Hence, we are going down in the steepest direction

- The second step moves a distance α down with said rate of change. ⟩ going down in the steepest direction
- The parameter α may be fixed
  ↳ so α is called the learning rate

or we can perform a line search where we try several values of α to find the one that decreases loss the most.

- At the minimum of the loss, the gradient is 0. So the parameters stop changing.
  ↳ In practice, we monitor the gradient magnitude & stop training when it gets too small.

Linear Regression Example:

- Consider a model $f[x,\phi]$ that maps a scalar input x to scalar output y with parameters $\phi = [\phi_0, \phi_1]^T$

$$y = f[x,\phi]$$
$$= \phi_0 + \phi_1 x$$

- Given a dataset $\{x_i, y_i\}$ with I input/output pairs, we choose the least squares loss function:

$$L[\phi] = \sum_{i=1}^{I} l_i = \sum_{i=1}^{I} (f[x_i,\phi] - y_i)^2$$
$$= \sum_{i=1}^{I} (\phi_0 + \phi_1 x_i - y_i)^2$$

where $l_i = (\phi_0 + \phi_1 x_i - y_i^2)$ is the individual contribution to the loss from the i^{th} training example.

- The derivative of the loss function with respect to the parameters can be written as:

$$\frac{\partial L}{\partial \phi} = \frac{\partial}{\partial \phi} \sum_{i=1}^{I} l_i = \sum_{i=1}^{I} \frac{\partial l_i}{\partial \phi}$$

where:

$$\frac{\partial l_i}{\partial \phi} = \begin{bmatrix} \partial l_i / \partial \phi_0 \\ \partial l_i / \partial \phi_1 \end{bmatrix} = \begin{bmatrix} 2(\phi_0 + \phi_1 x_i - y_i) \\ 2x_i(\phi_0 + \phi_1 x_i - y_i) \end{bmatrix}$$

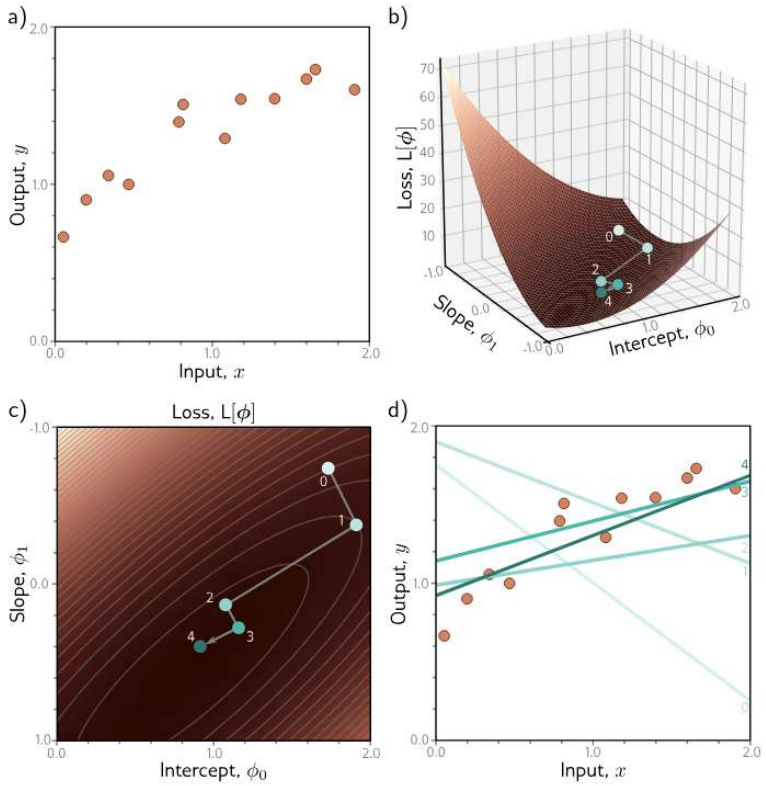


Figure 6.1 Gradient descent for the linear regression model. a) Training set of $I = 12$ input/output pairs $\{x_i, y_i\}$. b) Loss function showing iterations of gradient descent. We start at point 0 and move in the steepest downhill direction until we can improve no further to arrive at point 1. We then repeat this procedure. We measure the gradient at point 1 and move downhill to point 2 and so on. c) This can be visualized better as a heatmap, where the brightness represents the loss. After only four iterations, we are already close to the minimum. d) The model with the parameters at point 0 (lightest line) describes the data very badly, but each successive iteration improves the fit. The model with the parameters at point 4 (darkest line) is already a reasonable description of the training data.

Gabor Model Example:

- Loss functions for linear regression problems are convex!
  - ↳ Implies that regardless of starting conditions, the training will never fail.
- Loss functions for non-linear models are non-convex.
- Toy example: Gabor Model:

$$f[x,\phi] = \sin[\phi_0 + 0.06\,\phi_1 x_1] \cdot \exp\left[-\frac{(\phi_0 + 0.06\,\phi_1 x_1)^2}{32}\right]$$

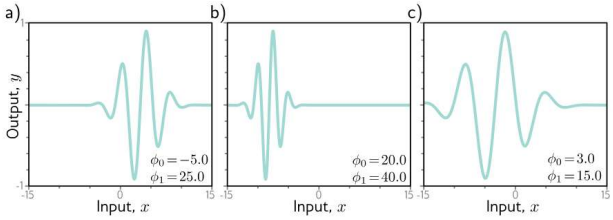


Figure 6.2 Gabor model. This nonlinear model maps scalar input x to scalar output y and has parameters $\phi = [\phi_0, \phi_1]^T$. It describes a sinusoidal function that decreases in amplitude with distance from its center. Parameter $\phi_0 \in \mathbb{R}$ determines the position of the center. As ϕ_0 increases, the function moves left. Parameter $\phi_1 \in \mathbb{R}^+$ squeezes the function along the x-axis relative to the center. As ϕ_1 increases, the function narrows. a–c) Model with different parameters. (Interactive figure)

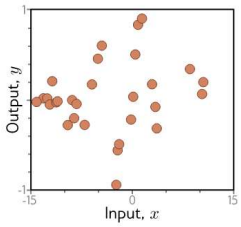


Figure 6.3 Training data for fitting the Gabor model. The training dataset contains 28 input/output examples $\{x_i, y_i\}$. These data were created by uniformly sampling $x_i \in [-15, 15]$, passing the samples through a Gabor model with parameters $\phi = [0.0, 16.6]^T$, and adding normally distributed noise.

- The loss function used is the least squares loss function:

$$L[\phi] = \sum_{i=1}^{I} (f[x_i, \phi] - y_i)^2$$

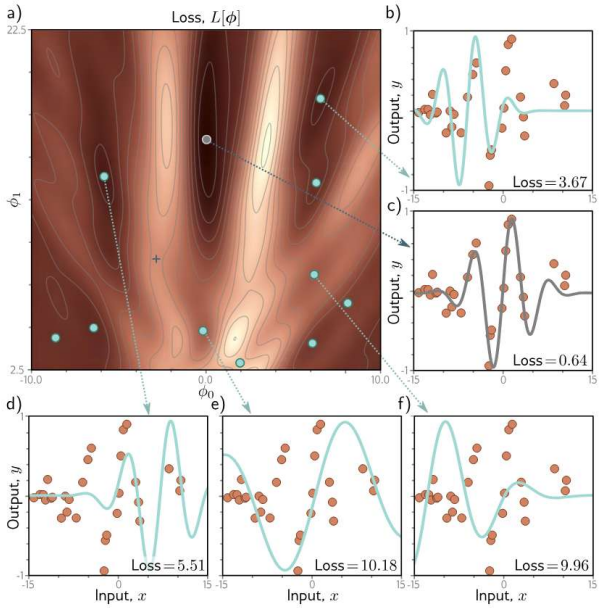


Figure 6.4 Loss function for the Gabor model. a) The loss function is non-convex, with multiple local minima (cyan circles) in addition to the global minimum (gray circle). It also contains saddle points where the gradient is locally zero, but the function increases in one direction and decreases in the other. The blue cross is an example of a saddle point; the function decreases as we move horizontally in either direction but increases as we move vertically. b–f) Models associated with the different minima. In each case, there is no small change that decreases the loss. Panel (c) shows the global minimum, which has a loss of 0.64. (Interactive figure)

↳ Numerous local minima

↳ Saddle Points

- With gradient descent, there is no guarantee our training will end at the global minima.

- With gradient descent, there is no guarantee our training will end at the global minima.

6.2: Stochastic Gradient Descent:

- Using gradient descent to find the global optima of a high dimensional loss function is challenging.
  - ↳ can find a minimum, but no way to tell if it's the global minimum / good at all.
- The Problem! The final destination of the gradient descent algorithm is determined only by the starting point.
  - ↳ Stochastic gradient descent tries to fix this by adding noise to the gradient at each step.
  - ↳ Still moves downward on average, but on any given iteration the direction chosen is not necessarily in the steepest downhill direction.
  - ↳ Might not even be downhill; SGD can move temporarily uphill ; "jump" from one "valley" of the loss function to another.

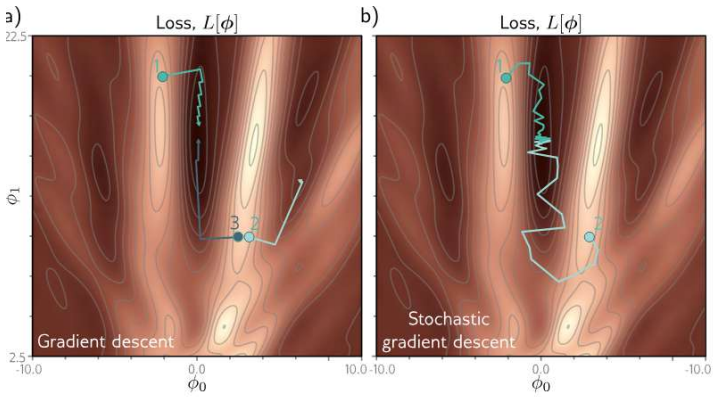


Figure 6.5 Gradient descent vs. stochastic gradient descent. a) Gradient descent with line search. As long as the gradient descent algorithm is initialized in the right "valley" of the loss function (e.g., points 1 and 3), the parameter estimate will move steadily toward the global minimum. However, if it is initialized outside this valley (e.g., point 2), it will descend toward one of the local minima. b) Stochastic gradient descent adds noise to the optimization process, so it is possible to escape from the wrong valley (e.g., point 2) and still reach the global minimum.

Batches & Epochs:

- To introduce randomness:
  - ↳ At each iteration, the algorithm chooses a random subset of the training dat
  - ↳ Computes the gradient from these examples alone. → Known as a "batch".
- The update rule for the model parameters ϕ_t at iteration t is:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \sum_{i \in B_t} \frac{\partial L_i[\phi_t]}{\partial \phi}$$

Pair in the current batch

ϕ_{t+1} ϕ_t $-\alpha \sum_{i \in B_t} \frac{\partial L_i}{\partial \phi}$

- B_t is the set containing the indices of the input/output pairs in the current batch
- L_i is the loss due to the i^{th} pair
- α is the learning rate; determines the distance moved [alongside gradient magnitude]

- The batches are usually drawn without replacement.
  - ↳ once it uses all the data, it starts sampling from the full training dataset again.
- A single pass through the entire training dataset is called an epoch.
- A batch can be as small as 1 example or as large as the whole traing dataset.
  - ↳ called full-batch gradient descent. Is identical to regular gradient descent.

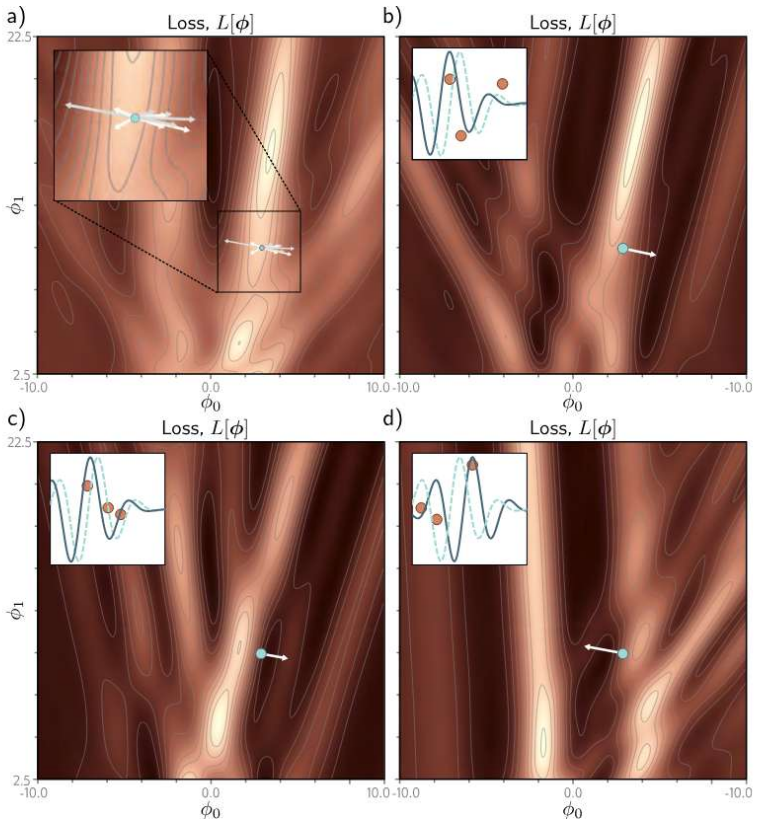


Figure 6.6 Alternative view of SGD for the Gabor model with a batch size of three. a) Loss function for the entire training dataset. At each iteration, there is a probability distribution of possible parameter changes (inset shows samples). These correspond to different choices of the three batch elements. b) Loss function for one possible batch. The SGD algorithm moves in the downhill direction on this function for a distance that is determined by the learning rate and the local gradient magnitude. The current model (dashed function in inset) changes to better fit the batch data (solid function). c) A different batch creates a different loss function and results in a different update. d) For this batch, the algorithm moves *downhill* with respect to the batch loss function but *uphill* with respect to the global loss function in panel (a). This is how SGD can escape local minima.

Alternative interpretation:

SGD computes the gradient of a different loss function at each iteration.

↳ Loss depends on model & training data

↳ Thus it differs from each batch

- In this view, SGD does deterministic gradient descent on a constantly changing loss function.

Properties of SGD:

1) Updates are sensible, even if not optimal.

2) Training examples all contribute equally
   ↳ Due to batch selection without replacement

3) Less computationally expensive to compute the gradient of a subset of the data.

4) It can escape local minima

5) Less likely to get stuck near saddle points.

6) Some evidence that suggests SGD finds parameters that generalize well to unseen data.

- In practice, SGD is applied with a learning rate schedule.
  - ↳ Learning rate α starts at a high value
  - ↳ Decreased by by a constant factor every N epochs.
  - ↳ The logic: Early in training we want to explore the parameter space & figure out sensible regions.
    Later on, we are roughly in the right place & want to fine tune parameters.

6.3: Momentum:

- A common modification to SGD is to add a momentum term.
- Update parameters with a weighted combination of the gradient computed from the current batch & the direction moved in the previous step:

$$m_{t+1} \leftarrow \beta \cdot m_t + (1-\beta) \sum_{i \in B_t} \frac{\partial L_i[\phi_t]}{\partial \phi}$$

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot m_{t+1}$$

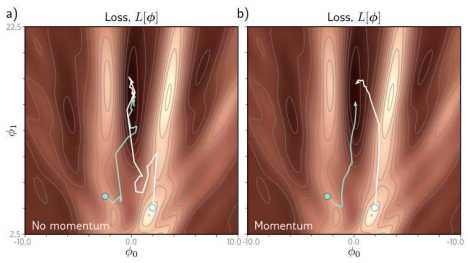
* m_t is the momentum [drives the update at iteration t]

* $\beta \in [0,1)$ controls the degree to which the gradient is smoothed over time.

- The recursive formulation means the gradient step is an infinite weighted sum of all the previous gradients where the weights get smaller as we move back in time.

- The effective learning rate increases if these gradients are aligned over multiple iterations.
  - ↳ Decreases if the gradient direction repeatedly changes as the terms in the sum cancel out.

- Overall effect: Smoother trajectory & reduced oscillatory behavior in valleys.



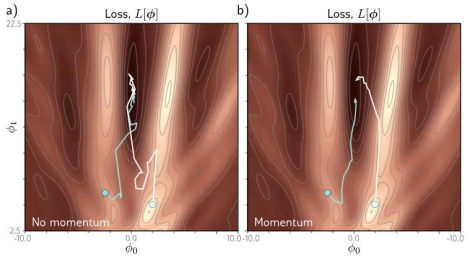


Figure 6.7 Stochastic gradient descent with momentum. a) Regular stochastic descent takes a very indirect path toward the minimum. b) With a momentum term, the change at the current step is a weighted combination of the previous change and the gradient computed from the batch. This smooths out the trajectory and increases the speed of convergence.

Nesterov Accelerated Momentum:

- The momentum term can be considered a coarse prediction of where the SGD Algorithm will move next.
- Nesterov Accelerated momentum computes the gradients at this predicted point rather than at the current point:

$$m_{t+1} \leftarrow \beta \cdot m_t + (1-\beta) \sum_{i \in B_t} \frac{\partial L_i[\phi_t - \alpha \cdot \beta \cdot m_t]}{\partial \phi}$$

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot m_{t+1}$$

* Gradients are now evaluated at $\phi_t - \alpha \cdot \beta \cdot m_t$.
* Can be interpreted as: the gradient term now corrects the path provided by momentum alone.

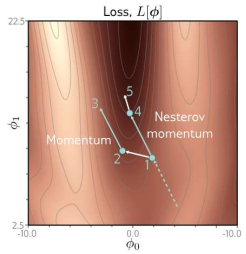


Figure 6.8 Nesterov accelerated momentum. The solution has traveled along the dashed line to arrive at point 1. A traditional momentum update measures the gradient at point 1, moves some distance in this direction to point 2, and then adds the momentum term from the previous iteration (i.e., in the same direction as the dashed line), arriving at point 3. The Nesterov momentum update first applies the momentum term (moving from point 1 to point 4) and then measures the gradient and applies an update to arrive at point 5.

6.4: Adam:

- Gradient descent w/ fixed step size has undesirable properties:
  - ↳ Makes large adjustments to parameters associated w/ large gradients ↳ May want to be cautious
  - ↳ Makes small adjustments to parameters associated w/ small gradients ↳ May want to explore further
- When the gradient of the loss surface is much steeper in one direction than another, hard to choose a learning rate that:
  - i) Makes good progress in both directions
  - ii) is stable
- Idea: we normalize the gradients so we move a fixed distance in each direction.
- To do this:

  Measure the gradient m_{t+1} & pointwise squared gradient v_{t+1}:

$$m_{t+1} \leftarrow \frac{\partial L[\phi_t]}{\partial \phi}$$

$$v_{t+1} \leftarrow \left(\frac{\partial L[\phi_t]}{\partial \phi}\right)^2$$

  Then apply the update rule:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \frac{m_{t+1}}{\sqrt{v_{t+1}} + \epsilon}$$

  ↳ Square & division are point-wise.

  ↳ Small constant to prevent dividing by 0.

$\sqrt{v_{t+1}} + \epsilon$

↳ Small constant to prevent dividing by 0.

- Result is the algorithm moves a fixed distance α along each coordinate.
  ↳ Direction is determined by whichever way is downhill.

- Won't converge unless exactly on the gradient.

- Adaptive Moment Estimation (Adam) takes the idea ; adds momentum to both the estimate of the gradient ; the squared gradient:

$$m_{t+1} \leftarrow \beta \cdot m_t + (1-\beta)\frac{\partial L[\phi_t]}{\partial \phi}$$

$$v_{t+1} \leftarrow \gamma \cdot v_t + (1-\gamma)\left(\frac{\partial L[\phi_t]}{\partial \phi}\right)^2$$

*β, γ are momentum coefficients.

- Momentum is equivalent to taking a weighted average over the history of the gradients.
- At the start, all the measurements are near 0, so we modify the statistics using the rule:

$$\tilde{m}_{t+1} \leftarrow \frac{m_{t+1}}{1-\beta^{t+1}}$$

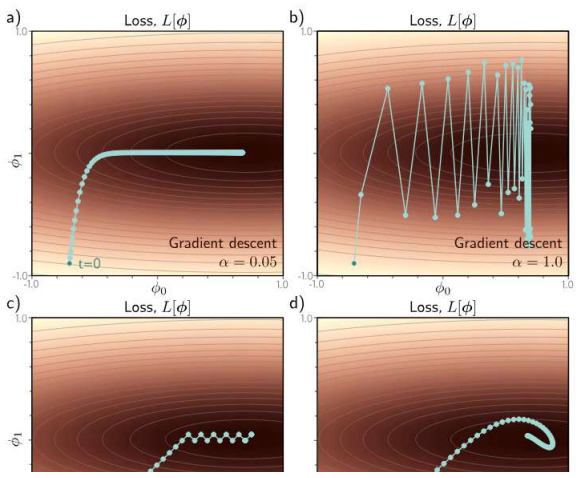
$$\tilde{v}_{t+1} \leftarrow \frac{v_{t+1}}{1-\gamma^{t+1}}$$

- β, γ are in the range $[0, 1)$.
  ↳ So with each time step, the terms with exponents $t+1$ get closer to 1.

- We update the parameters like before:

$$\phi_{t+1} \leftarrow \phi_t - \alpha \cdot \frac{\tilde{m}_{t+1}}{\sqrt{\tilde{v}_{t+1}} + \epsilon}$$

- The result is an algorithm that can converge to the overall minimum ; makes good progress in every direction of the parameter space.



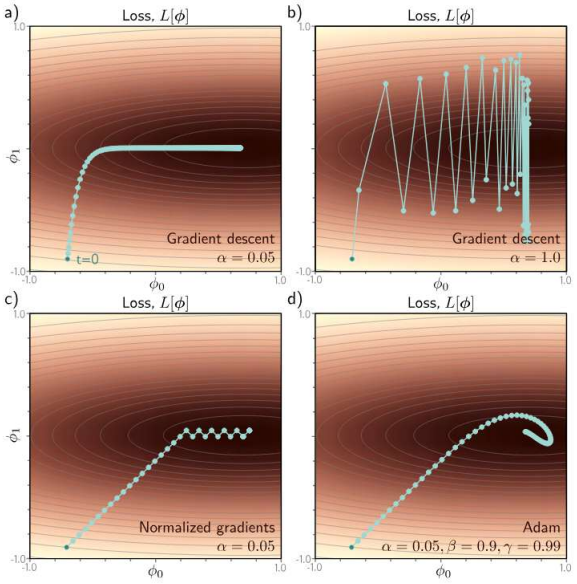


Figure 6.9 Adaptive moment estimation (Adam). a) This loss function changes quickly in the vertical direction but slowly in the horizontal direction. If we run full-batch gradient descent with a learning rate that makes good progress in the vertical direction, then the algorithm takes a long time to reach the final horizontal position. b) If the learning rate is chosen so that the algorithm makes good progress in the horizontal direction, it overshoots in the vertical direction and becomes unstable. c) A straightforward approach is to move a fixed distance along each axis at each step so that we move downhill in both directions. This is accomplished by normalizing the gradient magnitude and retaining only the sign. However, this does not usually converge to the exact minimum but instead oscillates back and forth around it (here between the last two points). d) The Adam algorithm uses momentum in both the estimated gradient and the normalization term, which creates a smoother path.

- Adam is usually used in a stochastic setting where gradients are computed from mini-batches:

$$m_{t+1} \leftarrow \beta \cdot m_t + (1-\beta) \sum_{i \in \mathcal{B}_t} \frac{\partial l_i[\phi_t]}{\partial \phi}$$

$$v_{t+1} \leftarrow \gamma \cdot v_t + (1-\gamma) \sum_{i \in \mathcal{B}_t} \left(\frac{\partial l_i[\phi_t]}{\partial \phi} \right)^2$$

- Adam is less sensitive to initial learning parameters in practice

6.5: Training Algorithm Hyperparameters:

- Choice of learning algorithm
- Batch size
- Learning rate schedule
- Momentum coefficients

6.6: Summary:

- Training models:
  - ↳ Find parameters ϕ that minimize loss $L[\phi]$

- Training Models:
  - Find Parameters ϕ that minimize loss $L[\phi]$
  - Gradient descent
  - For non-linear loss functions, it may have local minima & saddle points. Gradient descent fails in both situations.
  - SGD helps mitigate these problems by adding noise.
  - Adding momentum to SGD makes convergence more efficient
  - Adam to address learning rate problems.
- The ideas in this chapter apply to any model.